

```

"""
ARKHEIA-CPS – OIA-PEM Housing Service
Layer 2: Generate fairness pattern
expectations for HOUSING contracts.
Evaluates rent-to-income, move-in costs,
fees, rent increases, clauses, eviction.
"""

from __future__ import annotations
from dataclasses import dataclass, field
from decimal import Decimal
from typing import List

from app.schemas.housing import
HousingContractIn
from app.schemas.common import
PEMEvaluation

# — Thresholds

```

```

RENT_INCOME_SAFE      = Decimal("0.30")
RENT_INCOME_WARN      = Decimal("0.35")
RENT_INCOME_HIGH      = Decimal("0.45")
RENT_INCOME_SEVERE    = Decimal("0.55")

MOVEIN_SAFE_MONTHS     = Decimal("2.0")
MOVEIN_HIGH_MONTHS     = Decimal("2.5")
MOVEIN_EXTREME_MONTHS  = Decimal("3.0")

```

```
RENT_INCREASE_SAFE    = Decimal("8.0")
RENT_INCREASE_HIGH    = Decimal("10.0")
RENT_INCREASE_PRED    = Decimal("15.0")

APP_FEE_SAFE          = Decimal("75")
APP_FEE_HIGH          = Decimal("150")
ADMIN_FEE_SAFE        = Decimal("150")
ADMIN_FEE_HIGH        = Decimal("300")
SECURITY_SAFE_MONTHS  = Decimal("1.0")
SECURITY_HIGH_MONTHS  = Decimal("2.0")
RECURRING_PRED_MO     = Decimal("75")
EARLY_TERM_PRED_MOS   = Decimal("3.0")

# Illegal clause keywords – automatic FAIL
ILLEGAL_CLAUSE_PATTERNS = [
    "waive habitability",
    "no right to sue",
    "waive right to sue",
    "no jury trial",
    "waive notice",
    "surrender right",
    "landlord not liable",
    "landlord has no responsibility",
    "no refund",
    "non-refundable deposit",
    "entry without notice",
    "landlord may enter at any time",
    "tenant waives",
    "confess judgment",
    "cognovit",
]

@dataclass
```

```

class OIAPEMHousingResult:
    evaluations: List[PEMEvaluation] =
field(default_factory=list)
    dimension_flags: dict =
field(default_factory=dict)
    illegal_clauses: List[str] =
field(default_factory=list)
    predatory_flags: List[str] =
field(default_factory=list)

class OIAPEMHousingService:
    """
    OIA-PEM pattern engine for
    HOUSING/lease contracts.
    """

    def evaluate(self, payload:
HousingContractIn) -> OIAPEMHousingResult:
        result = OIAPEMHousingResult()

        result.evaluations +=
self._eval_rent_to_income(payload)
        result.evaluations +=
self._eval_movein_costs(payload)
        result.evaluations +=
self._eval_fees(payload)
        result.evaluations +=
self._eval_rent_increase(payload)
        result.evaluations +=
self._eval_late_fee(payload)
        result.evaluations +=
self._eval_grace_period(payload)
        result.evaluations +=

```

```

self._eval_early_termination(payload)
    result.evaluations +=
self._eval_automatic_renewal(payload)
    result.evaluations +=
self._eval_eviction(payload)
    result.evaluations +=
self._eval_recurring_fees(payload)

    # Illegal clause scan
    illegal, predatory =
self._scan_clauses(payload)
    result.illegal_clauses = illegal
    result.predatory_flags = predatory
    if illegal:

result.evaluations.append(PMEvaluation(

pattern_family="HABITABILITY",
pattern_key="ILLEGAL_CLAUSES",

actual_value=Decimal(str(len(illegal))),
status="FAIL",
    explanation=f"
{len(illegal)} illegal clause(s) detected:
{'; '.join(illegal[:3])}"
    ))

    # Dimension flags for RTCSA scoring
    result.dimension_flags = {
        "affordability_fail":
any(e.pattern_family == "AFFORDABILITY" and
e.status == "FAIL" for e in
result.evaluations),
        "affordability_warn":

```

```

any(e.pattern_family == "AFFORDABILITY" and
e.status == "WARN" for e in
result.evaluations),
    "fee_fail":
any(e.pattern_family == "FEE" and
e.status == "FAIL" for e in
result.evaluations),
    "fee_warn":
any(e.pattern_family == "FEE" and
e.status == "WARN" for e in
result.evaluations),
    "lease_term_fail":
any(e.pattern_family == "LEASE_TERM" and
e.status == "FAIL" for e in
result.evaluations),
    "lease_term_warn":
any(e.pattern_family == "LEASE_TERM" and
e.status == "WARN" for e in
result.evaluations),
    "habitability_fail":
any(e.pattern_family == "HABITABILITY" and
e.status == "FAIL" for e in
result.evaluations),
    "eviction_fail":
any(e.pattern_family == "EVICTION" and
e.status == "FAIL" for e in
result.evaluations),
    "eviction_warn":
any(e.pattern_family == "EVICTION" and
e.status == "WARN" for e in
result.evaluations),
    "has_illegal_clauses":
len(illegal) > 0,
    }

```

```

        return result

# — Rent-to-Income


---




---



    def _eval_rent_to_income(self, p:
HousingContractIn) -> List[PEMEvaluation]:
        net = p.consumer.net_monthly_income
or p.consumer.reported_monthly_income
        if not net:
            return [PEMEvaluation(

pattern_family="AFFORDABILITY",
pattern_key="RENT_TO_INCOME",
            status="WARN",
explanation="Income not provided – rent
affordability cannot be assessed."
            )]

        ratio = p.base_monthly_rent /
Decimal(str(net))

        if ratio > RENT_INCOME_SEVERE:
            status = "FAIL"
            expl = (f"Rent-to-income ratio
of {float(ratio)*100:.1f}% is severe. "
                    f"At this level the
tenant will likely face housing
instability.")
        elif ratio > RENT_INCOME_HIGH:
            status = "FAIL"
            expl = (f"Rent-to-income ratio

```

```

of {float(ratio)*100:.1f}% is high risk. "
        f"Recommended maximum
is 30-35%.")
    elif ratio > RENT_INCOME_WARN:
        status = "WARN"
        expl = f"Rent-to-income ratio
of {float(ratio)*100:.1f}% is elevated
(above 35%)."
    else:
        status = "OK"
        expl = f"Rent-to-income ratio
of {float(ratio)*100:.1f}% is within
acceptable range."

    return [PEMEvaluation(
        pattern_family="AFFORDABILITY",
        pattern_key="RENT_TO_INCOME",
        expected_min=Decimal("0"),
        expected_max=RENT_INCOME_WARN,
        actual_value=ratio,
        status=status, explanation=expl,
    )]

```

— Move-In Costs

```

def _eval_movein_costs(self, p:
HousingContractIn) -> List[PEMEvaluation]:
    rent = p.base_monthly_rent
    total = p.security_deposit_amount
    if p.first_month_rent_due:
        total += rent
    if p.last_month_rent_due:

```

```

        total += rent
        total += p.application_fee_amount +
p.admin_fee_amount
        total += sum(Decimal(str(f.amount))
for f in p.other_upfront_fees)

        months_equiv = total / rent if rent
> 0 else Decimal("0")

        if months_equiv >
MOVEIN_EXTREME_MONTHS:
            status = "FAIL"
            expl = (f"Total move-in cost of
${total:,.2f} ({float(months_equiv):.1f}×
rent) "
                    f"is extreme. Amounts
above 3× monthly rent are predatory in most
jurisdictions.")
            elif months_equiv >
MOVEIN_HIGH_MONTHS:
                status = "WARN"
                expl = (f"Total move-in cost of
${total:,.2f} ({float(months_equiv):.1f}×
rent) "
                        f"is above the 2.5×
guideline. This is a high upfront burden.")
            else:
                status = "OK"
                expl = f"Total move-in cost of
${total:,.2f} ({float(months_equiv):.1f}×
rent) is acceptable."

        return [PEMEvaluation(
            pattern_family="FEE",

```

```

pattern_key="TOTAL_MOVEIN_COST",

expected_max=MOVEIN_SAFE_MONTHS,
        actual_value=months_equiv,
status=status, explanation=expl,
    )]

# — Individual Fees


---




---



def _eval_fees(self, p:
HousingContractIn) -> List[PEMEvaluation]:
    results = []

    # Application fee
    if p.application_fee_amount >
APP_FEE_HIGH:
        results.append(PEMEvaluation(
            pattern_family="FEE",
pattern_key="APPLICATION_FEE",
            expected_max=APP_FEE_SAFE,
actual_value=p.application_fee_amount,
            status="FAIL",
            explanation=f"Application
fee of ${p.application_fee_amount:,.2f}
exceeds predatory threshold of
${APP_FEE_HIGH:,.2f}."
        ))
    elif p.application_fee_amount >
APP_FEE_SAFE:
        results.append(PEMEvaluation(
            pattern_family="FEE",
pattern_key="APPLICATION_FEE",

```

```

        expected_max=APP_FEE_SAFE,
actual_value=p.application_fee_amount,
        status="WARN",
        explanation=f"Application
fee of ${p.application_fee_amount:,.2f} is
above typical range ($25-$75)."
    ))

    # Admin fee
    if p.admin_fee_amount >
ADMIN_FEE_HIGH:
        results.append(PEMEvaluation(
            pattern_family="FEE",
pattern_key="ADMIN_FEE",

expected_max=ADMIN_FEE_SAFE,
actual_value=p.admin_fee_amount,
            status="FAIL",
            explanation=f"Admin fee of
${p.admin_fee_amount:,.2f} exceeds
predatory threshold of
${ADMIN_FEE_HIGH:,.2f}."
        ))
    elif p.admin_fee_amount >
ADMIN_FEE_SAFE:
        results.append(PEMEvaluation(
            pattern_family="FEE",
pattern_key="ADMIN_FEE",

expected_max=ADMIN_FEE_SAFE,
actual_value=p.admin_fee_amount,
            status="WARN",
            explanation=f"Admin fee of
${p.admin_fee_amount:,.2f} is above the

```

```

$150 guideline."
    ))

    # Security deposit (by months of
rent)
    rent = p.base_monthly_rent
    if rent > 0:
        dep_months =
p.security_deposit_amount / rent
        if dep_months >
SECURITY_HIGH_MONTHS:

results.append(PMEvaluation(
                    pattern_family="FEE",
pattern_key="SECURITY_DEPOSIT",

expected_max=SECURITY_SAFE_MONTHS,
actual_value=dep_months,
                    status="FAIL",
                    explanation=(f"Security
deposit of
${p.security_deposit_amount:,.2f} "
                                f"
({float(dep_months):.1f}× rent) exceeds 2×
– likely illegal in many states.")
                    ))
        elif dep_months >
SECURITY_SAFE_MONTHS:

results.append(PMEvaluation(
                    pattern_family="FEE",
pattern_key="SECURITY_DEPOSIT",

expected_max=SECURITY_SAFE_MONTHS,

```

```

actual_value=dep_months,
                status="WARN",
                explanation=f"Security
deposit is {float(dep_months):.1f}× monthly
rent. 1× is typical."
            ))

        return results

# — Rent Increase


---




---



    def _eval_rent_increase(self, p:
HousingContractIn) -> List[PEMEvaluation]:
        cap =
p.rent_increase_cap_percentage
        if cap is None:
            if p.rent_increase_clause_text:
                return [PEMEvaluation(

pattern_family="LEASE_TERM",
pattern_key="RENT_INCREASE_CAP",
                status="WARN",
                explanation="Rent
increase clause exists but no cap
percentage is specified. This gives the
landlord unlimited increase authority."
            )]
        return []

    if cap > RENT_INCREASE_PRED:
        status = "FAIL"
        expl = f"Rent increase cap of

```

```

{float(cap):.1f}% is predatory (above
15%)."
        elif cap > RENT_INCREASE_HIGH:
            status = "WARN"
            expl = f"Rent increase cap of
{float(cap):.1f}% is high (above 10%
guideline)."
        else:
            status = "OK"
            expl = f"Rent increase cap of
{float(cap):.1f}% is within expected range
(3-8%)."

        return [PEMEvaluation(
            pattern_family="LEASE_TERM",
            pattern_key="RENT_INCREASE_CAP",

            expected_max=RENT_INCREASE_SAFE,
            actual_value=cap,
            status=status,
            explanation=expl,
        )]

# — Late Fee

```

```

def _eval_late_fee(self, p:
HousingContractIn) -> List[PEMEvaluation]:
    rent = p.base_monthly_rent
    fee = p.late_fee_amount

    if p.late_fee_type == "percentage"
and p.late_fee_percentage:

```

```

        # Convert to dollar amount for
comparison
        fee = rent *
p.late_fee_percentage / 100

        if fee > rent * Decimal("0.10"):
            status = "FAIL"
            expl = (f"Late fee of
${fee:,.2f} exceeds 10% of monthly rent
(${rent:,.2f}). "
                    f"Fees above this
threshold are predatory in most
jurisdictions.")
            elif fee > rent * Decimal("0.05"):
                status = "WARN"
                expl = f"Late fee of
${fee:,.2f} is above the 5% guideline."
            else:
                status = "OK"
                expl = f"Late fee of
${fee:,.2f} is within acceptable range."

        return [PEMEvaluation(
            pattern_family="FEE",
pattern_key="LATE_FEE",
            expected_max=rent *
Decimal("0.05"),
            actual_value=fee,
status=status, explanation=expl,
        )]

# — Grace Period

```

```

def _eval_grace_period(self, p:
HousingContractIn) -> List[PEMEvaluation]:
    if p.grace_period_days == 0:
        return [PEMEvaluation(

pattern_family="LEASE_TERM",
pattern_key="GRACE_PERIOD",
            actual_value=Decimal("0"),
status="WARN",
            explanation="No grace
period provided. Most fair leases allow 3-5
days before late fees apply."
        )]
    if p.grace_period_days < 3:
        return [PEMEvaluation(

pattern_family="LEASE_TERM",
pattern_key="GRACE_PERIOD",
            expected_min=Decimal("3"),
actual_value=Decimal(str(p.grace_period_day
s)),
            status="WARN",
            explanation=f"Grace period
of {p.grace_period_days} day(s) is below
the 3-day recommended minimum."
        )]
    return [PEMEvaluation(
        pattern_family="LEASE_TERM",
pattern_key="GRACE_PERIOD",

actual_value=Decimal(str(p.grace_period_day
s)), status="OK",
            explanation=f"Grace period of

```

```

{p.grace_period_days} days is adequate."
    )]

    # — Early Termination
    —

    def _eval_early_termination(self, p:
HousingContractIn) -> List[PEMEvaluation]:
        rent = p.base_monthly_rent
        penalty = None

        if p.early_termination_penalty_type
== "months_of_rent" and
p.early_termination_months:
            penalty = rent *
p.early_termination_months
            months =
p.early_termination_months
        elif
p.early_termination_penalty_amount:
            penalty =
p.early_termination_penalty_amount
            months = penalty / rent if
rent > 0 else Decimal("0")
        else:
            return []

        if months > EARLY_TERM_PRED_MOS:
            status = "FAIL"
            expl = (f"Early termination
penalty of ${penalty:,.2f}
({float(months):.1f}× rent) "
f"is predatory. Amounts

```

```

above 3× monthly rent are excessive.")
    elif months > Decimal("2.0"):
        status = "WARN"
        expl = f"Early termination
penalty of ${penalty:,.2f}
({float(months):.1f}× rent) is elevated."
    else:
        status = "OK"
        expl = f"Early termination
penalty of ${penalty:,.2f} is within
acceptable range."

    return [PEMEvaluation(
        pattern_family="LEASE_TERM",
        pattern_key="EARLY_TERMINATION_PENALTY",

        expected_max=EARLY_TERM_PRED_MOS,
        actual_value=months,
        status=status,
        explanation=expl,
    )]

```

— Automatic Renewal

—

```

def _eval_automatic_renewal(self, p:
HousingContractIn) -> List[PEMEvaluation]:
    if not p.automatic_renewal:
        return []
    notice =
p.automatic_renewal_notice_days
    if not notice or notice < 30:
        return [PEMEvaluation(

```

```

pattern_family="LEASE_TERM",
pattern_key="AUTO_RENEWAL_NOTICE",

actual_value=Decimal(str(notice or 0)),
status="FAIL",
        explanation=(
            f"Lease auto-renews but
only provides {notice or 0} days notice. "
            f"Automatic renewal
with fewer than 30 days notice is a high-
risk pattern."
        )
    ]
    return [PEMEvaluation(
        pattern_family="LEASE_TERM",
        pattern_key="AUTO_RENEWAL_NOTICE",

        actual_value=Decimal(str(notice)),
        status="OK",
        explanation=f"Automatic renewal
with {notice} days notice is acceptable."
    )]

# — Eviction

```

```

def _eval_eviction(self, p:
HousingContractIn) -> List[PEMEvaluation]:
    results = []
    trigger =
p.late_payment_eviction_trigger_days
    notice = p.eviction_notice_days

```

```

        grace    = p.grace_period_days or 0

        if trigger and notice:
            if trigger < grace:

results.append(PMEvaluation(

pattern_family="EVICTION",
pattern_key="EVICTION_BEFORE_GRACE",
                status="FAIL",
                explanation=(f"Eviction
can be triggered after {trigger} days but
the grace period "
                                f"is
{grace} days – eviction trigger is before
grace period ends.")
                ))
            if notice < 3:

results.append(PMEvaluation(

pattern_family="EVICTION",
pattern_key="EVICTION_NOTICE_TOO_SHORT",

expected_min=Decimal("3"),
actual_value=Decimal(str(notice)),
                status="FAIL",
                explanation=f"Eviction
notice of {notice} day(s) is below the
legally required minimum in most states."
                ))
            elif notice < 5:

results.append(PMEvaluation(

```

```

pattern_family="EVICTION",
pattern_key="EVICTION_NOTICE_SHORT",

actual_value=Decimal(str(notice)),
status="WARN",

                                explanation=f"Eviction
notice of {notice} days is short. Most
tenant-protective leases provide 5-30
days."
                                ))

```

```

    return results

```

```

# — Recurring Fees

```

```

    def _eval_recurring_fees(self, p:
HousingContractIn) -> List[PEMEvaluation]:
        mandatory_monthly = sum(
            Decimal(str(f.amount)) for f in
p.recurring_fees
            if f.mandatory and f.frequency
== "monthly"
        )
        if mandatory_monthly >
RECURRING_PRED_MO:
            return [PEMEvaluation(
                pattern_family="FEE",
pattern_key="MANDATORY_RECURRING_FEES",

expected_max=RECURRING_PRED_MO,
actual_value=mandatory_monthly,

```

```

        status="FAIL",
        explanation=(f"Mandatory
recurring fees total
${mandatory_monthly:,.2f}/mo - "
                    f"exceeds the
${RECURRING_PRED_MO:,.2f}/mo predatory
threshold.")
    )]
    return []

```

— Clause Scanner

```

    def _scan_clauses(self, p:
HousingContractIn) -> tuple[List[str],
List[str]]:
        illegal    = []
        predatory  = []

        all_text = " ".join([
            c.text.lower() for c in
p.lease_clauses
        ]) + [
            p.termination_clause_text or
            "",
            p.eviction_clause_text or "",
            p.maintenance_clause_text or
            "",
            p.dispute_resolution_clause_text or "",
        ])

        for pattern in

```

```
ILLEGAL_CLAUSE_PATTERNS:
    if pattern in all_text:
        illegal.append(pattern)

    # Predatory: arbitration required +
tenant pays fees
    if p.arbitration_required and
p.dispute_fee_sharing_terms:
        if "tenant" in
(p.dispute_fee_sharing_terms or
"".lower():

predatory.append("Arbitration required with
tenant responsible for fees")

    # Predatory: non-refundable
security deposit
    if p.security_deposit_type ==
"non_refundable":
        predatory.append("Non-
refundable security deposit")

    return illegal, predatory
```